

Experiment 1

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. Hash Generation using SHA-256:

- o Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

2. Target Hash Generation with Nonce:

- o Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

3. Proof-of-Work Puzzle Solving:

- o Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

4. Merkle Tree Construction:

- o Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output:

Program #1: Python program that uses the hashlib library to create the hash of a given string:

```
import hashlib

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()

    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))

    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()

    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

```
Enter a string: Saniya Kadam
Hash: 733a5a1f0a356d42917d5944469d29a2426a6a526ec5a8df17141e7fce22d631
```

Program #2: Program to generate required target hash with input string and nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)
```

```
Enter a string: Saniya Kadam
Enter the nonce: 7
Hash Code: ec6ac968d72a35b257803da27441f222f51f1ff5e257a7e0aec6045cc562c171
```

Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string

```
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)

        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce

        nonce += 1

# Get user input
input_string = "NEHA KADAM"
num_zeros = 1

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
```

```
Hash: 0b9b36526a069342d44dec80c153f93c7aa6ab10e3c9590ceaf9fcbdf10d9cff
Input String: NEHA KADAM
Leading Zeros: 1
Expected Nonce: 12
```

Program 4: Generating Merkle Tree for given set of Transactions

```

import hashlib

def sha256_hash(data):
    return hashlib.sha256(data.encode('utf-8')).hexdigest()

def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None

    # Hash all transactions before combining them
    transactions = [sha256_hash(transaction) for transaction in transactions]

    print("Initial Transaction Hashes:")
    for i, transaction_hash in enumerate(transactions):
        print(f"Hash {i + 1}: {transaction_hash}")

    # Construct the Merkle Tree
    while len(transactions) > 1:

        # If odd number of transactions, duplicate the last one
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])

        new_transactions = []

```

```

        for i in range(0, len(transactions), 2):

            # Combine two hashes
            combined = transactions[i] + transactions[i + 1]

            # Hash the combined hashes
            hash_combined = hashlib.sha256(
                combined.encode('utf-8')
            ).hexdigest()

            new_transactions.append(hash_combined)

            # Print intermediate hash
            print(f"Intermediate Hash: {hash_combined}")

        transactions = new_transactions

    return transactions[0]

```

```

# Sample Transactions
transactions = [
    "Siya -> Sanjana : $200",
    "Sanjana -> Saniya : $500",
    "Saniya -> Viraj : $100",
    "Viraj -> Siya : $300",
    "Roo -> Sanjana : $50"
]

# Construct the Merkle Tree
merkle_root = build_merkle_tree(transactions)

# Print the final Merkle Root
print("\nMerkle Root:", merkle_root)

```

```

Initial Transaction Hashes:
Hash 1: 6c4ea4613abdaee0dbe4c95eb7198555ec8132af1a43e9da977a6689c75216f
Hash 2: 15efde1fe51326c21ce718c5bb2184576931225d2e9718aa08e38d321cfc3964
Hash 3: 2ff61004d564b227956408f1ce5b80f6085848903ad966977876d5da054e4f6e
Hash 4: 6a490f69ff04880bd2ed986ff00a025e8cbf93b1d4763e4a5a5cdc7e9f83f270
Hash 5: 4eea3bdf788754a7c4bd29990ae8ce584a22a3115bf4724173e9af1167b07732
Intermediate Hash: e36d122d8f2c7c73390787a984c999480f0b695df2552eb030e911700427202b
Intermediate Hash: dcdd3ac3161005f38eba70a0156ba2957d36447a8658aabcb60a0ea532d40975
Intermediate Hash: 0bfd4dd7ddab65da457d6670db8d13335e7594934443fb1709d50c8de40b977d
Intermediate Hash: b4e0175bd5546750520f7c57cb63b539628028aeddb06571ae7fa75e6e64165
Intermediate Hash: b84d3e9f8f6d7e7e5428f45852550377136aa393819502174c625522c84582b0
Intermediate Hash: ab77ac5b2807e868c6325723820ab9d1f18e287a6017da0dd86a625c1a8e5796

Merkle Root: ab77ac5b2807e868c6325723820ab9d1f18e287a6017da0dd86a625c1a8e5796

```

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.